

Universidad Tecnológica de Pereira
Facultad de Ingenierías
Programa de Ingeniería de Sistemas y Computación
HPC

Implementación Autómata Celular para el Tráfico Vehicular usando MPI

Presentado por:

Juan Esteban Cardona Blandón
Jhoan Esteban Corrales Rodríguez
Sebastian Perdomo
Juana Valentina Martínez

Profesor:

Ramiro Andrés Barrios Valencia

Pereira, Colombia

2 de junio de 2026

Índice

1. Marco Teórico	3
1.1. Computación de Alto Rendimiento (HPC)	3
1.2. Programación Secuencial y Concurrente	3
1.3. Sistemas de Memoria Compartida	3
1.4. Jerarquía de Caché, Líneas de Caché y Localidad de Memoria	4
1.5. Métricas de Rendimiento en Computación Secuencial	4
1.6. Métricas de Rendimiento en Computación Paralela	5
1.7. Autómatas Celulares	6
1.8. Modelo de Nagel-Schreckenberg	7
1.9. Transición de Fase Libre-Congestionado	8
1.10. Intensidad Aritmética y Modelo Roofline	8
1.11. MPI	9
1.12. Network File System (NFS)	10
2. Marco Conceptual	11
2.1. Características de los Nodos de Ejecución	11
2.2. Características del software	11
3. Desarrollo de la implementación	12
3.1. Modelo del autómata celular de tráfico	12
3.2. Velocidad media y estado estacionario	12
3.3. Criterio de convergencia del calentamiento	13
3.4. Modelo de distribución de datos	13
3.5. Estructura de datos distribuida	13
3.6. Condiciones de frontera periódicas en el entorno distribuido	14
3.7. Intercambio de halos no bloqueante	14
3.8. Kernel distribuido con solapamiento comunicación-cómputo	15
3.9. Propiedad SPMD del calentamiento	15
3.10. Temporización y salida	15
3.11. Arquitectura de módulos	16
3.12. Variantes de compilación	16
3.13. Validador de corrección distribuido	16
3.14. Despliegue en el clúster AWS	17
3.15. Diseño del script de benchmark	17

4. Pruebas	18
4.1. Resultados algoritmo secuencial	18
4.2. Resultados de la implementación con MPI sin optimizaciones	22
4.3. Resultados de la implementación con MPI con optimizaciones por compilador	25
4.4. Comparación	29
5. Conclusiones	30

1. Marco Teórico

1.1. Computación de Alto Rendimiento (HPC)

La Computación de Alto Rendimiento (HPC, por sus siglas en inglés) consiste en la agregación de potencia de cálculo a través de arquitecturas paralelas para resolver problemas científicos y tecnológicos de extrema complejidad. Esta disciplina permite ejecutar algoritmos y modelados matemáticos que resultarían inabarcables para un sistema tradicional debido a restricciones de tiempo y recursos de hardware (Hager & Wellein, 2010). Al dividir cargas masivas de trabajo en instrucciones más pequeñas que se procesan simultáneamente, HPC maximiza la velocidad de procesamiento de datos y la obtención de resultados en la ingeniería (Patterson & Hennessy, 2017).

1.2. Programación Secuencial y Concurrente

La programación secuencial es el paradigma clásico de desarrollo donde las instrucciones de un algoritmo se ejecutan de forma estrictamente lineal, una tras otra, sobre un único núcleo de procesamiento. En este modelo tradicional, el límite de rendimiento está directamente dictado por la frecuencia de reloj del hardware. Como respuesta a estas limitaciones físicas, la programación concurrente estructura el software en múltiples hilos o tareas independientes que hacen progreso de forma superpuesta en el tiempo. Cuando estas tareas operan de manera simultánea en una arquitectura de hardware multinúcleo, la concurrencia se traduce en paralelismo físico, lo cual permite dividir la carga de trabajo y reducir drásticamente el tiempo de ejecución en problemas de cómputo intensivo (Ben-Ari, 2006).

1.3. Sistemas de Memoria Compartida

En el ámbito de la computación paralela, el modelo de memoria compartida establece que múltiples hilos de ejecución (como los creados por la librería Pthreads) operan dentro de un mismo espacio de direccionamiento lógico. A nivel de arquitectura, esto significa que todos los núcleos del procesador pueden acceder de forma directa y simultánea a las mismas estructuras de datos residentes en la memoria principal. Para problemas matriciales, este modelo resulta excepcionalmente eficiente frente a los sistemas de memoria distribuida, ya que evita la necesidad de copiar y transmitir los datos de las matrices de origen repetidamente entre procesos. Aunque este paradigma exige control riguroso para evitar condiciones de carrera, en operaciones intrínsecamente independientes como el cálculo individual de las celdas de una matriz resultado, su implementación maximiza la eficiencia computacional sin sobrecarga de sincronización (Silberschatz et al., 2018).

1.4. Jerarquía de Caché, Líneas de Caché y Localidad de Memoria

El rendimiento de una aplicación de alto rendimiento (HPC) no depende exclusivamente de la capacidad matemática del procesador, sino también del acceso al subsistema de memoria. Para mitigar la alta latencia térmica y física de la memoria RAM, los procesadores modernos integran una jerarquía de memoria ultrarrápida cercana a los núcleos, conocida como memoria Caché, habitualmente dividida en niveles (L1, L2 y L3). La unidad mínima de transferencia de información entre la RAM principal y la memoria caché no es un byte aislado, sino un bloque continuo de datos denominado línea de caché (*cache line*), que en las arquitecturas modernas suele constar de 64 bytes (Drepper, 2007).

Esta transferencia por bloques está fundamentada en el principio de localidad espacial de memoria, un concepto empírico que dicta que si un programa accede a una dirección de memoria, es inminentemente probable que acceda a las direcciones adyacentes a continuación. En lenguajes de programación como C, las matrices bidimensionales se almacenan en la memoria RAM de manera contigua organizadas por filas (*row-major order*). Durante la multiplicación matricial clásica iterativa, la segunda matriz se recorre inevitablemente por columnas; este patrón de acceso ortogonal rompe por completo la localidad espacial. Al intentar leer datos saltando grandes bloques de memoria, los valores necesarios no se encuentran en la línea de caché cargada, provocando fallos de caché (*cache misses*) que obligan al procesador a detenerse (*stall*) mientras busca los datos nuevamente en la lenta memoria RAM, lo cual penaliza severamente el desempeño del algoritmo (Patterson & Hennessy, 2017).

1.5. Métricas de Rendimiento en Computación Secuencial

El análisis de un algoritmo secuencial requiere establecer una línea base absoluta sobre la cual evaluar cualquier mejora algorítmica o de hardware. La métrica fundamental empírica es el Tiempo de Ejecución (*Wall-clock time*), que representa el tiempo físico real transcurrido desde el inicio hasta la finalización del programa. Sin embargo, dado que este valor fluctúa dependiendo de la interferencia del sistema operativo y los accesos a memoria, la arquitectura de computadores recurre a métricas de rendimiento intrínseco (*throughput*) (Patterson & Hennessy, 2017).

Para cargas de trabajo de alto rendimiento algebraico como la multiplicación de matrices, la métrica estandarizada predominante es el volumen de Operaciones de Punto Flotante por Segundo (FLOPS, por sus siglas en inglés). Esta medida cuantifica el rendimiento matemático bruto del procesador y se define mediante la siguiente relación:

$$\text{FLOPS} = \frac{\text{Operaciones Aritméticas Totales}}{T_{\text{ejecución}}} \quad (1)$$

Para el algoritmo clásico iterativo que multiplica dos matrices cuadradas de dimensión $n \times n$, el número total de operaciones de punto flotante se establece analíticamente en $2n^3$ (contabilizando una instrucción de multiplicación y una de suma por cada iteración del bucle más interno). Al contrastar los FLOPS experimentales obtenidos en la ejecución secuencial contra el rendimiento teórico máximo (*Peak FLOPS*) que el fabricante del procesador garantiza para un solo núcleo, es posible diagnosticar ineficiencias de memoria o cuellos de botella antes de recurrir a la paralelización (Hager & Wellein, 2010).

Adicionalmente, el rendimiento de la ejecución en un único núcleo se evalúa a nivel microarquitectónico mediante el IPC (Instrucciones por Ciclo de reloj). Esta métrica indica la eficiencia con la que el código compilado logra mantener ocupada la segmentación (*pipeline*) del procesador, minimizando las detenciones o burbujas producidas por dependencias de datos en la ejecución lineal (Yi et al., 2020).

1.6. Métricas de Rendimiento en Computación Paralela

Para cuantificar el éxito de una implementación paralela y su eficiencia frente a la ejecución secuencial, la literatura científica emplea métricas matemáticas estandarizadas. La métrica fundamental es el *Speedup* o Aceleración (S_p), la cual mide la ganancia de velocidad relativa. Se define formalmente como:

$$S_p = \frac{T_1}{T_p} \quad (2)$$

donde T_1 representa el tiempo de ejecución del algoritmo secuencial óptimo sobre un único núcleo, y T_p es el tiempo de ejecución del algoritmo paralelo empleando p unidades de procesamiento (hilos o procesos). Derivada de esta métrica se encuentra la Eficiencia (E_p), que evalúa el grado de aprovechamiento de los recursos físicos, indicando qué fracción del tiempo los procesadores realizan trabajo útil sin quedar ociosos:

$$E_p = \frac{S_p}{p} = \frac{T_1}{p \cdot T_p} \quad (3)$$

El desempeño asintótico de estos sistemas está regido por dos leyes analíticas que abordan la escalabilidad desde diferentes perspectivas teóricas. La **Ley de Amdahl** evalúa la *escalabilidad fuerte*, demostrando que el *Speedup* máximo de un sistema está estrictamente limitado por la fracción de código que no se puede paralelizar. Asumiendo un tamaño de problema fijo, la ley se expresa como:

$$S_{\text{Amdahl}} \leq \frac{1}{(1 - f) + \frac{f}{p}} \quad (4)$$

donde f es la fracción del programa que es paralelizable y $(1 - f)$ es la porción estrictamente secuencial (como la inicialización de memoria mediante `malloc` o la lectura/escritura en disco). Según este postulado matemático, incluso si el número de procesadores p tiende a infinito, la aceleración máxima nunca superará $1/(1 - f)$. Esto explica el rendimiento decreciente observado cuando el uso de múltiples hilos lógicos en un procesador alcanza su límite físico y no aporta mejoras significativas (Patterson & Hennessy, 2017).

En contraste, la **Ley de Gustafson-Barsis** describe la *escalabilidad débil*. Esta ley argumenta que, en la práctica profesional, los ingenieros no buscan resolver un problema fijo más rápido, sino que utilizan el mayor poder de cómputo para resolver problemas de un tamaño sustancialmente mayor en el mismo marco de tiempo (Gustafson, 1988). Su modelo se define como:

$$S_{\text{Gustafson}} = (1 - f) + p \cdot f \quad (5)$$

Bajo la visión de Gustafson, a medida que la dimensión de las matrices crece de forma masiva, la cantidad de operaciones paralelas f (multiplicaciones flotantes) aumenta drásticamente, haciendo que la fracción secuencial inicial $(1 - f)$ pierda peso relativo. Esto mitiga el cuello de botella pronosticado por Amdahl, permitiendo alcanzar un *Speedup* escalable y un uso altamente eficiente de los recursos al ajustar la carga de trabajo proporcionalmente al hardware.

1.7. Autómatas Celulares

Un autómata celular (AC) es un sistema dinámico discreto distribuido espacialmente en el que tanto el tiempo como el espacio se representan de forma discreta (Wolfram, 1984). Formalmente, un AC unidimensional se define como la tupla $\mathcal{A} = (\mathbb{Z}, Q, \mathcal{N}, f)$, donde $\mathbb{Z}$ es el espacio de celdas indexadas por los enteros, Q es un conjunto finito de estados posibles, $\mathcal{N} = \{-r, \dots, 0, \dots, r\}$ es la vecindad de radio r , y $f : Q^{2r+1} \rightarrow Q$ es la función de transición local. En cada paso de tiempo, esta función se aplica de manera sincrónica e idéntica a todas las celdas:

$$R^{t+1}(i) = f(R^t(i - r), \dots, R^t(i), \dots, R^t(i + r)).$$

La propiedad que habilita directamente el cómputo paralelo es esta actualización sincrónica: el estado nuevo de la celda i depende exclusivamente del estado de su vecindad en el instante t , sin que intervengan cambios ocurridos durante el mismo paso. En consecuencia, todas las celdas son computacionalmente independientes entre sí dentro de un mismo paso de tiempo, lo que hace del modelo de AC una estructura naturalmente paralelizable sin condiciones de carrera entre hilos.

Wolfram clasificó los AC elementales unidimensionales, aquellos con $|Q| = 2$ y $r = 1$, que generan 256 reglas distintas, en cuatro clases de comportamiento asintótico (Wolfram, 1984). La clase I agrupa autómatas que convergen a un estado homogéneo uniforme independientemente de las condiciones iniciales, análogo a un punto límite en sistemas continuos. La clase II produce estructuras periódicas o estacionarias, análoga a ciclos límite. La clase III exhibe comportamiento caótico semejante al de atractores extraños. La clase IV genera patrones complejos localizados con interacciones no triviales, y se ha conjeturado que es capaz de computación universal, por lo que propiedades de su comportamiento a tiempo infinito son indecidibles.

1.8. Modelo de Nagel-Schreckenberg

El modelo de Nagel y Schreckenberg (NaSch), publicado en 1992, fue el primer autómata celular diseñado explícitamente para simular el flujo vehicular en una vía de un solo carril (Nagel & Schreckenberg, 1992). El modelo opera sobre un arreglo unidimensional de L celdas con condiciones de frontera periódicas, donde cada celda puede estar vacía u ocupada por un vehículo con velocidad entera $v \in \{0, 1, \dots, v_{\text{máx}}\}$. La dinámica de cada paso de tiempo $t \rightarrow t + 1$ se define mediante cuatro reglas aplicadas en paralelo a todos los vehículos simultáneamente:

1. Aceleración: $v_n \leftarrow \min(v_n + 1, v_{\text{máx}})$.
2. Frenado por interacción: $v_n \leftarrow \min(v_n, d_n - 1)$, donde d_n es el número de celdas vacías frente al vehículo n .
3. Aleatorización: con probabilidad p , si $v_n > 0$, entonces $v_n \leftarrow v_n - 1$.
4. Movimiento: el vehículo n avanza v_n celdas.

El parámetro $p \in [0, 1]$ modela la conducta imperfecta del conductor humano y fue la primera formalización del origen estocástico de los atascos espontáneos sin causa aparente en la vía.

La implementación desarrollada en este trabajo corresponde al caso límite determinista con $v_{\text{máx}} = 1$ y $p = 0$. Bajo estas condiciones, la regla de aceleración es trivial dado que todo vehículo alcanza $v = 1$ en el mismo paso, y la aleatorización desaparece, reduciéndose la dinámica a una sola condición binaria: un vehículo avanza si y solo si la celda inmediatamente frente a él estaba vacía en el instante t . Este caso particular es algebraicamente equivalente a la Regla 184 de los AC elementales y constituye el caso base analíticamente tratable del modelo NaSch, estudiado explícitamente por los autores originales (Nagel & Schreckenberg, 1992). La

simplificación es justificada porque preserva el comportamiento colectivo esencial del modelo, la transición entre flujo libre y congestionado, mientras elimina los grados de libertad que no son relevantes para el análisis de rendimiento paralelo que motiva este trabajo.

1.9. Transición de Fase Libre-Congestionado

El experimento de barrido de densidades tiene por objetivo observar el cambio cualitativo de comportamiento del sistema en función de la densidad vehicular $\rho = N_{\text{cars}}/N$. Para densidades bajas, la mayoría de los vehículos encuentra espacio libre y la velocidad media tiende a $\bar{v} \approx 1$, correspondiente al régimen de flujo libre. Para densidades altas, los vehículos se bloquean mutuamente y $\bar{v} \rightarrow 0$, lo que define el régimen congestionado. Esta transición entre fases de comportamiento cualitativamente distinto fue uno de los resultados centrales reportados por Nagel y Schreckenberg (Nagel & Schreckenberg, 1992), quienes la denominaron transición de flujo laminar a ondas de arranque-parada.

En el caso determinista con $v_{\text{máx}} = 1$ y $p = 0$, la transición es abrupta y ocurre exactamente en $\rho = 0,5$. Para $\rho \leq 0,5$, todos los vehículos pueden moverse en cada paso y $\bar{v} = 1$; para $\rho > 0,5$, el flujo queda limitado por la densidad de espacios vacíos y $\bar{v} = (1 - \rho)/\rho$. La representación canónica de este fenómeno es el diagrama fundamental, que grafica el flujo vehicular $q = \rho \cdot \bar{v}$ en función de ρ . Para el modelo implementado, el diagrama fundamental adopta una forma triangular simétrica, conocida en la literatura como función tent, con un máximo de $q_{\text{máx}} = 0,5$ en $\rho = 0,5$. Este diagrama es la referencia estándar para validar modelos de tráfico en la literatura, y justifica tanto el rango de densidades elegido en el experimento como el registro de la velocidad media $\bar{v}$ como variable de salida principal del simulador.

1.10. Intensidad Aritmética y Modelo Roofline

El modelo Roofline, propuesto por Williams, Waterman y Patterson (Williams et al., 2009), es un modelo de rendimiento visual que relaciona el desempeño alcanzable de un kernel de cómputo con dos parámetros del hardware: el rendimiento de cómputo pico π (en operaciones por segundo) y el ancho de banda de memoria pico β (en bytes por segundo). La variable que caracteriza al kernel es la intensidad aritmética:

$$I = \frac{\text{operaciones ejecutadas}}{\text{bytes transferidos desde/hacia memoria principal}}.$$

El desempeño alcanzable queda acotado superiormente por:

$$P \leq \min(\pi, \beta \cdot I).$$

Un kernel es memory-bound si $I < \pi/\beta$, en cuyo caso el cuello de botella es el ancho de banda de memoria y no la capacidad de cómputo; es compute-bound si $I > \pi/\beta$. El punto de cresta (ridge point) en la coordenada $I = \pi/\beta$ marca la frontera entre ambos regímenes.

Para el autómata celular de tráfico implementado, cada celda requiere tres lecturas ($R^t(i-1)$, $R^t(i)$, $R^t(i+1)$) y una escritura ($R^{t+1}(i)$), junto con aproximadamente dos operaciones lógicas para evaluar la regla de transición. Representando cada celda como un entero de un byte, la transferencia de memoria por celda es de 4 bytes y el número de operaciones es de alrededor de 2, lo que produce una intensidad aritmética $I \approx 0,4$ op/byte. Este valor se sitúa por debajo del punto de cresta de cualquier arquitectura multicore contemporánea, clasificando el kernel del autómata como claramente memory-bandwidth bound (Williams et al., 2009).

Esta caracterización tiene una implicación directa para la escalabilidad paralela de la implementación: al incrementar el número de hilos, la demanda agregada de ancho de banda de la memoria compartida se satura antes de que la capacidad de cómputo adicional pueda aprovecharse. En consecuencia, el speedup esperado con p hilos alcanzará una saturación anticipada determinada no por el overhead de sincronización ni por la fracción serial del programa (Ley de Amdahl), sino por el límite físico del ancho de banda de memoria de la arquitectura subyacente.

1.11. MPI

MPI (*Message Passing Interface*) es un estándar para programación paralela mediante paso de mensajes, diseñado principalmente para sistemas de memoria distribuida. Su propósito es ofrecer una interfaz portable, eficiente y ampliamente soportada para desarrollar aplicaciones paralelas en lenguajes como C, C++ y Fortran (MPI Forum, 2021; Snir et al., 1998).

Una de las características fundamentales de MPI es que el programador controla explícitamente la descomposición del problema, la distribución de datos y la comunicación entre procesos. Esto permite construir aplicaciones con un modelo SPMD (*Single Program, Multiple Data*), en el cual múltiples procesos ejecutan el mismo programa pero trabajan sobre datos distintos (Gropp et al., 2014).

En MPI, los procesos se organizan dentro de comunicadores, que definen conjuntos de procesos autorizados para intercambiar mensajes. Cada proceso dentro de un comunicador

posee un rango (*rank*) que lo identifica de forma única y permite establecer el origen o destino de la comunicación. Esta abstracción facilita tanto las comunicaciones punto a punto como las operaciones colectivas.

Entre las operaciones más utilizadas de MPI se encuentran el envío y recepción de mensajes entre procesos, así como las comunicaciones colectivas como difusión (*broadcast*), dispersión (*scatter*), reunión (*gather*) y reducción (*reduce*). Estas primitivas permiten implementar de manera estructurada algoritmos paralelos donde los datos deben repartirse, procesarse localmente y recombinarse al final de la ejecución (MPI Forum, 2021).

En el contexto de la multiplicación de matrices, MPI es particularmente adecuado porque el problema puede descomponerse por filas o bloques, de manera que cada proceso calcula una parte independiente de la matriz resultado. No obstante, el rendimiento final depende de la relación entre el tamaño del problema, el costo de comunicación y la eficiencia del núcleo de cómputo local. Por esta razón, las implementaciones MPI suelen requerir además optimización algorítmica y cuidadosa atención al acceso a memoria (Pacheco, 1997; Snir et al., 1998).

1.12. Network File System (NFS)

El Network File System (NFS) es un protocolo de sistema de archivos distribuido que permite a varios nodos acceder a archivos almacenados de forma remota como si estuvieran disponibles localmente en el sistema operativo. Su principal ventaja es que proporciona un mecanismo transparente de compartición de datos, de manera que los procesos pueden leer y escribir archivos ubicados en un servidor central sin requerir una lógica explícita de transferencia de archivos en la aplicación (Sandberg et al., 1985; Stoner, Terry et al., 1995).

En entornos de computación paralela y distribuida, NFS resulta útil para compartir binarios, datos de entrada y resultados entre múltiples nodos de un clúster. Esta característica simplifica el despliegue de aplicaciones MPI, ya que todos los procesos pueden acceder a la misma estructura de directorios y ejecutar el mismo binario sin necesidad de copiar manualmente los archivos en cada máquina (Storm et al., 1988; Tanenbaum & Steen, 2015).

Una propiedad importante de NFS es que el acceso a archivos remotos introduce costos adicionales de red y de sincronización frente a un sistema de archivos local. Aunque el cliente percibe la operación como una lectura o escritura convencional, la solicitud debe transmitirse al servidor y esperar respuesta, lo que puede afectar el tiempo total de ejecución si el volumen de accesos es alto o si el sistema se utiliza simultáneamente por varios procesos (Stone & Hennessy, 1995; Tanenbaum & Steen, 2015).

En un clúster MPI, este comportamiento tiene implicaciones directas sobre el rendimiento. Si los archivos de entrada se leen desde NFS al inicio de la ejecución, la sobrecarga puede

ser aceptable; sin embargo, si la aplicación depende de accesos frecuentes o si varios procesos realizan lecturas concurrentes sobre archivos grandes, la latencia de red y el ancho de banda del sistema de almacenamiento pueden convertirse en un cuello de botella. Por esta razón, el uso de NFS debe analizarse no solo como una comodidad operativa, sino también como un factor que influye en las mediciones experimentales (Stone & Hennessy, 1995; Terry, 1995).

2. Marco Conceptual

2.1. Características de los Nodos de Ejecución

Para este proyecto se usaron para los nodos (Head Node y Compute Node) instancias de AWS EC2 m7i-flex.large, las cuales cuentan con las siguientes características:

- Instancia utilizada: AWS EC2 m7i-flex.large
- Procesador: Intel Xeon Scalable (Sapphire Rapids, 4ta generación)
- Cantidad de vCPUs: 2
- Cantidad de núcleos: 1
- Cantidad de subprocesos/hilos: 2
- Frecuencia del procesador: hasta 3.2 GHz
- Memoria RAM: 8 GiB DDR5
- Arquitectura: x86_64 (64 bits)
- Almacenamiento: EBS Only
- Ancho de banda de red: hasta 12.5 Gbps
- Sistema operativo: Linux

2.2. Características del software

Se puede encontrar el código fuente del proyecto en el siguiente repositorio de GitHub:
<https://github.com/Juanes7222/HPC-CellularAutomaton>

3. Desarrollo de la implementación

En esta sección se presenta la construcción del simulador de tráfico unidimensional usando paralelismo con MPI. El objetivo principal fue mantener separadas la lógica del autómata, la distribución del estado y el intercambio de información entre procesos, de manera que las diferencias observadas en rendimiento dependan realmente del número de procesos y de la estrategia de compilación, y no de cambios en el algoritmo. Esta separación también facilita la verificación de corrección y hace más clara la comparación entre la versión secuencial y la distribuida.

3.1. Modelo del autómata celular de tráfico

El sistema representa una carretera circular dividida en N celdas. Cada celda puede estar vacía o contener un carro, y en cada paso de tiempo cada carro intenta avanzar a la siguiente posición si esa celda está libre. Si la celda siguiente está ocupada, el carro permanece en su lugar. Esta dinámica captura un comportamiento simple pero suficiente para estudiar flujo vehicular, conservación de vehículos y efectos de congestión.

La regla de evolución se expresa de forma local: el nuevo estado de una celda depende únicamente de su contenido actual y del contenido de sus vecinas inmediata izquierda y derecha. Gracias a esto, el problema se presta muy bien al paralelismo, porque cada posición puede actualizarse de forma independiente una vez que se conocen los valores de frontera correspondientes.

En la implementación, cada celda se almacena como un entero de 8 bits sin signo (`uint8_t`). Esto permite trabajar con un formato compacto y eficiente en memoria. La actualización del estado se traduce entonces a operaciones lógicas elementales sobre bytes, evitando estructuras más pesadas y manteniendo el kernel de cómputo muy liviano. Además, como el cálculo del siguiente estado se realiza sobre dos arreglos distintos, uno para el estado actual y otro para el siguiente, no hay dependencias entre iteraciones del bucle principal.

3.2. Velocidad media y estado estacionario

Una de las magnitudes más importantes del modelo es la velocidad media de los carros. En cada paso, esta velocidad corresponde a la fracción de vehículos que lograron avanzar. Como el sistema conserva el número total de carros, esta medida resume de manera sencilla qué tan fluido está el tráfico en un instante dado.

Cuando la simulación comienza desde una configuración aleatoria, el sistema atraviesa primero una etapa transitoria en la que los carros se reacomodan. Durante ese periodo,

la velocidad puede fluctuar bastante. Después de suficientes pasos, la dinámica tiende a estabilizarse y la velocidad converge a un valor representativo del régimen estacionario. Ese valor es el que se usa para reportar el comportamiento promedio del sistema durante la fase de medición.

3.3. Criterio de convergencia del calentamiento

La duración de la fase transitoria no es fija: depende de la densidad inicial y del tamaño de la carretera. Para evitar escoger un número de pasos de calentamiento arbitrario, se usa un criterio automático basado en la comparación de la velocidad promedio en dos ventanas consecutivas. Cuando la diferencia entre ambas ventanas cae por debajo de un umbral pequeño, se considera que el sistema ya alcanzó un comportamiento estable.

Este criterio se complementa con dos protecciones prácticas. Primero, se exige un mínimo de pasos antes de evaluar la convergencia, con el fin de evitar falsas detecciones al inicio de la simulación. Segundo, se fija un límite máximo de calentamiento para impedir que el proceso se prolongue indefinidamente si la estabilización tarda más de lo esperado. En conjunto, esto permite adaptar el calentamiento al tamaño del problema sin perder control sobre el tiempo total de ejecución.

3.4. Modelo de distribución de datos

Para paralelizar la simulación, la carretera se reparte entre P procesos MPI en bloques contiguos. Cada proceso recibe un segmento continuo de celdas, con una distribución muy equilibrada, la diferencia entre el bloque más grande y el más pequeño nunca supera una celda. Esto es importante porque el trabajo por celda es uniforme, así que repartir de esta forma evita desbalances innecesarios.

El estado global se construye inicialmente en el proceso de rango cero y luego se distribuye a los demás procesos mediante `MPI_Scatterv`. Esta estrategia simplifica la inicialización y permite que todos los rangos empiecen a trabajar con una parte coherente del mismo estado global. Para que la partición sea válida, el número de procesos no puede superar el número total de celdas; de lo contrario, algunos bloques quedarían vacíos y la topología circular del problema perdería sentido.

3.5. Estructura de datos distribuida

Cada proceso trabaja con una estructura que encapsula tanto el estado local como la información necesaria para comunicarse con sus vecinos. Esta estructura almacena los arreglos

con las celdas actuales y las celdas del siguiente paso, los tamaños globales y locales, el desplazamiento dentro de la carretera global y los rangos de los vecinos izquierdo y derecho. También guarda el número total de carros, que se calcula una sola vez al inicio y se usa después para normalizar la velocidad media.

Los búferes de datos se reservan alineados a 64 bytes, lo que favorece el acceso eficiente a memoria y mejora las posibilidades de vectorización por parte del compilador. Además, al intercambiar únicamente los punteros entre los arreglos actual y siguiente al terminar cada iteración, se evita copiar datos y se reduce el costo del ciclo de simulación.

3.6. Condiciones de frontera periódicas en el entorno distribuido

La carretera es circular, así que la última celda conecta con la primera. En la versión distribuida, esa continuidad debe respetarse aunque la información esté repartida entre procesos distintos. Por eso, en cada paso cada proceso necesita conocer una celda de su vecino izquierdo y una de su vecino derecho para poder actualizar correctamente sus posiciones de borde.

Estas celdas adicionales se manejan como halos. El halo izquierdo de un proceso proviene de la última celda del proceso anterior, mientras que el halo derecho proviene de la primera celda del proceso siguiente. En el caso de los extremos, la comunicación se cierra sobre sí misma: el primer proceso recibe información del último, y el último del primero. De este modo, la condición periódica del modelo se mantiene exactamente en el entorno MPI.

3.7. Intercambio de halos no bloqueante

Antes de actualizar su parte de la carretera, cada proceso intercambia sus valores de frontera con los vecinos. Para ello se utilizan operaciones no bloqueantes, específicamente `MPI_Irecv` y `MPI_Isend`. Esta decisión permite que la comunicación se inicie sin detener el proceso, de forma que mientras los mensajes viajan por la red se pueda avanzar con el cómputo del interior del bloque local.

El esquema de etiquetas se define cuidadosamente para evitar confusiones entre mensajes que van en sentidos opuestos. Cada proceso envía su primera celda a su vecino izquierdo y su última celda a su vecino derecho, y recibe de ellos los halos correspondientes. La simetría del protocolo hace que cada rango pueda interpretar de manera unívoca qué dato pertenece a cada frontera.

Este diseño no solo es correcto, sino también eficiente: reduce el tiempo ocioso y permite superponer parte de la latencia de comunicación con trabajo útil de cómputo.

3.8. Kernel distribuido con solapamiento comunicación-cómputo

La actualización de un paso completo se organiza en varias fases. Primero se publican los intercambios de halos. Luego se calcula el interior del bloque local, que no depende de datos externos y por tanto puede procesarse de inmediato. Después se espera a que lleguen las fronteras, se actualizan las celdas borde y finalmente se realiza una reducción global para obtener el total de carros que avanzaron durante ese paso.

Este orden tiene una intención clara, se quiere aprovechar al máximo el tiempo disponible mientras la red entrega los datos necesarios para los bordes. El interior del bloque suele ser la parte más grande del trabajo local, así que calcularlo mientras se completa la comunicación ayuda a esconder parte del costo de MPI.

La reducción global se hace con `MPI_Allreduce`, de modo que todos los procesos terminan con el mismo valor de velocidad media en cada paso. Esto es fundamental para que el criterio de convergencia pueda evaluarse de forma idéntica en todos los rangos sin agregar una comunicación adicional.

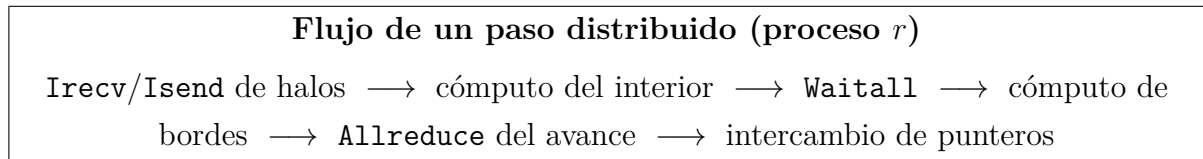


Figura 1: Secuencia general de un paso de simulación en la implementación MPI.

3.9. Propiedad SPMD del calentamiento

Como todos los procesos reciben el mismo valor de velocidad en cada paso, el criterio de convergencia se evalúa de la misma manera en todos ellos. Esto permite que la fase de calentamiento siga un comportamiento SPMD consistente, cada proceso toma la misma decisión en el mismo instante, sin necesidad de sincronizaciones adicionales para declarar que el sistema ya se estabilizó.

Esta propiedad simplifica mucho la lógica de control y evita complicaciones al finalizar la simulación. También reduce el riesgo de bloqueos, porque todos los procesos avanzan de forma coherente hacia la misma condición de término.

3.10. Temporización y salida

La medición de tiempo se realiza con `MPI_Wtime`, dejando fuera la fase de calentamiento para que el valor reportado refleje únicamente el comportamiento de la fase de medición. Como no todos los procesos terminan exactamente al mismo tiempo, el tiempo final se to-

ma como el máximo entre todos ellos, lo cual representa la duración real de la ejecución sincronizada.

El programa principal imprime una sola línea con el tiempo total y la velocidad promedio. Los mensajes auxiliares se envían a la salida de error para no interferir con el procesamiento automático de resultados. La interfaz de ejecución se mantiene simple, con los parámetros del tamaño de la carretera, densidad inicial, máximo de calentamiento y número de pasos de medición.

3.11. Arquitectura de módulos

La implementación se organizó en módulos con responsabilidades bien definidas. Un archivo se encarga del estado distribuido y de la distribución inicial de datos; otro concentra la lógica del kernel MPI y el intercambio de halos; un tercero coordina la ejecución general, la línea de comandos y la temporización; y un cuarto contiene los validadores distribuidos. Esta separación hace que el código sea más fácil de mantener y facilita la reutilización de componentes entre pruebas y experimentos.

Además, un archivo común define las etiquetas de comunicación y algunas funciones auxiliares para calcular los vecinos izquierdo y derecho. Con esto se evita duplicar lógica entre los módulos que manipulan la topología del anillo.

3.12. Variantes de compilación

Se construyeron dos binarios que comparten el mismo código fuente, pero difieren en las banderas de compilación. Uno actúa como versión de referencia y el otro como versión optimizada. Ambos se generan con `mpicc`, que funciona como un envoltorio sobre el compilador base y facilita el uso de MPI sin configurar manualmente las rutas de inclusión o enlace.

Las optimizaciones aplicadas buscan expresar el mismo kernel de cómputo, que es muy regular y tiene un perfil de acceso a memoria bastante predecible. Opciones como `-O3`, `-march=native` y `-flto` ayudan al compilador a vectorizar, incrustar funciones pequeñas y ajustar el código al procesador real en el que se ejecuta.

3.13. Validador de corrección distribuido

Para asegurar que la implementación paralela reproduce correctamente el comportamiento esperado, se desarrolló un validador distribuido con varios casos de prueba. Estos casos cubren situaciones extremas y también escenarios donde la frontera periódica es crucial. Entre ellos se incluyen una carretera vacía, una carretera completamente ocupada, un carro aislado, el

cruce correcto por el borde, la conservación del número de carros y la coincidencia con un estado calculado manualmente.

El validador reúne el estado global cuando es necesario y compara el resultado obtenido con el esperado. También verifica que todos los procesos lleguen a la misma conclusión, lo cual es importante para garantizar que la finalización de la simulación ocurra sin desincronizaciones.

3.14. Despliegue en el clúster AWS

Las pruebas se ejecutaron sobre un clúster de tres instancias EC2 tipo `c5.xlarge`. Todas las máquinas pertenecen a la misma red privada de AWS, lo que proporciona una comunicación estable y con latencias bajas entre nodos. Para este problema, donde cada proceso solo intercambia halos de un byte por paso, la infraestructura de red resulta más que suficiente.

El entorno compartido se montó mediante NFS, de modo que todos los nodos accedieran al mismo directorio con binarios, archivo de hosts y resultados. Esto simplifica la ejecución y evita inconsistencias entre máquinas. La compilación se realiza desde el nodo principal, y luego las pruebas se lanzan con `mpirun` usando afinidad por núcleo para minimizar interferencias.

La topología de comunicación entre procesos es muy ligera donde cada proceso solo conversa con sus dos vecinos inmediatos.

3.15. Diseño del script de benchmark

El script de benchmark funciona como un orquestador de experimentos. Su tarea es lanzar las ejecuciones con los parámetros correctos, recoger los resultados y guardarlos en un archivo CSV.

La columna `np` del CSV se usa para indicar el número de procesos MPI. Se usaron valores para `np` 2, 3, 4, 5 y 6, cada valor corresponde a una ejecución paralela con el número respectivo de procesos.

El techo de calentamiento se define de forma adaptativa para que escale con el tamaño del problema. Así se evita que instancias pequeñas desperdicien tiempo innecesario en calentamiento, mientras que las instancias grandes reciben suficiente margen para estabilizarse antes de comenzar la medición.

El archivo de resultados registra el binario usado, el número de procesos, el tamaño de la carretera, la densidad inicial, la repetición, el tiempo de ejecución y la velocidad promedio. El tiempo almacenado corresponde al proceso más lento, porque ese es el que determina la

duración real de una ejecución sincronizada y, por tanto, el valor correcto para el cálculo de speedup.

4. Pruebas

En esta sección se presentan las pruebas realizadas para evaluar el rendimiento de los algoritmos implementados. Se llevaron a cabo experimentos tanto en modo secuencial como en modo paralelo, utilizando diferentes tamaños de entrada y distintas configuraciones de optimización con el objetivo de comparar su impacto sobre el tiempo de ejecución.

Para la optimización por compilador se utilizó el siguiente conjunto de banderas:

- `-O3`: activa el nivel máximo de optimización general del compilador, aplicando transformaciones agresivas sobre el código para mejorar el rendimiento.
- `-march=native`: genera código optimizado específicamente para la arquitectura del procesador en el que se compila, aprovechando sus extensiones e instrucciones disponibles.
- `-funroll-loops`: desborda los ciclos cuando es posible, reduciendo el costo de control del bucle y mejorando la ejecución en regiones críticas.
- `-flto`: habilita la optimización en tiempo de enlace, permitiendo al compilador analizar y optimizar funciones y módulos completos de forma conjunta.
- `-fno-semantic-interposition`: permite al compilador asumir que las llamadas internas no serán interceptadas dinámicamente, lo que facilita optimizaciones más agresivas como inlining y desvirtualización.
- `-falign-loops=32`: alinea los bucles en fronteras de 32 bytes para mejorar el aprovechamiento de la caché de instrucciones y la eficiencia del flujo de ejecución.
- `-fomit-frame-pointer`: elimina el puntero de marco cuando no es necesario, liberando un registro adicional y reduciendo sobrecarga en funciones simples o críticas.

4.1. Resultados algoritmo secuencial

Estos son los resultados de cada una de las versiones secuenciales, implementadas para el Reto 2, ejecutadas en una de las máquinas de AWS antes descritas.

Serial Base						
Rep.	N=8K	N=64K	N=500K	N=2M	N=5M	N=10M
1	23,843	189,473	1.467,183	5.898,853	14.769,370	29.595,217
2	23,908	189,508	1.462,836	5.900,844	14.781,014	29.622,950
3	23,869	189,343	1.467,137	5.894,656	14.789,987	29.588,806
4	23,621	188,610	1.469,392	5.909,635	14.772,112	29.594,913
5	23,981	189,967	1.468,515	5.911,793	14.796,639	29.632,180
6	23,250	188,597	1.467,143	5.894,014	14.781,607	29.615,555
7	23,902	189,680	1.463,583	5.892,064	14.788,313	29.584,096
8	23,774	189,183	1.463,927	5.891,131	14.803,116	29.615,908
9	23,869	189,619	1.471,247	5.900,426	14.795,011	29.625,131
10	23,794	189,924	1.466,482	5.903,389	14.801,609	29.614,191
Promedio	23,781	189,390	1.466,745	5.899,681	14.787,878	29.608,895
Desv. Est.	0,199	0,454	2,534	6,713	11,093	15,894
CV (%)	0,84	0,24	0,17	0,11	0,08	0,05

Cuadro 1: Tiempos de ejecución del algoritmo secuencial sin optimizaciones

Serial con Optimización de Compilador						
Rep.	N=8K	N=64K	N=500K	N=2M	N=5M	N=10M
1	21,044	169,146	1.325,158	5.308,188	13.561,568	27.164,075
2	21,043	168,823	1.325,458	5.310,818	13.560,901	27.110,681
3	21,050	168,756	1.325,141	5.322,307	13.588,794	27.090,437
4	21,037	168,950	1.324,660	5.311,844	13.568,664	27.106,895
5	21,027	169,063	1.324,793	5.308,325	13.560,719	27.105,838
6	21,065	168,930	1.324,869	5.310,250	13.567,036	27.102,050
7	21,077	169,156	1.325,018	5.310,481	13.567,250	27.119,746
8	21,050	169,151	1.325,708	5.314,236	13.563,924	27.170,153
9	21,043	168,760	1.325,827	5.309,435	13.564,898	27.108,190
10	21,047	168,788	1.325,089	5.311,385	13.565,553	27.103,063
Promedio	21,048	168,952	1.325,172	5.311,727	13.566,931	27.118,113
Desv. Est.	0,013	0,158	0,364	3,902	7,745	25,503
CV (%)	0,06	0,09	0,03	0,07	0,06	0,09

Cuadro 2: Tiempos de ejecución del algoritmo secuencial con optimización por compilador

Serial con Optimización de Memoria						
Rep.	N=8K	N=64K	N=500K	N=2M	N=5M	N=10M
1	23,471	187,230	1.490,743	6.066,968	15.179,209	30.116,255
2	23,800	187,370	1.498,853	6.073,201	15.159,005	30.104,120
3	23,426	188,107	1.498,923	6.088,770	15.187,019	30.286,106
4	23,331	186,841	1.494,814	6.060,789	15.190,887	30.106,231
5	23,516	186,555	1.500,923	6.061,229	15.145,533	30.138,303
6	23,441	186,529	1.500,854	6.043,667	15.128,532	30.116,997
7	23,449	186,422	1.490,155	6.059,249	15.150,494	30.136,636
8	23,483	187,059	1.489,798	6.097,657	15.177,185	30.187,690
9	23,272	186,323	1.495,491	6.069,529	15.182,731	30.116,636
10	23,441	186,689	1.493,633	6.073,848	15.167,515	30.146,248
Promedio	23,463	186,912	1.495,419	6.069,491	15.166,811	30.145,522
Desv. Est.	0,132	0,517	4,107	14,560	19,361	52,380
CV (%)	0,56	0,28	0,27	0,24	0,13	0,17

Cuadro 3: Tiempos de ejecución del algoritmo secuencial con optimización por memoria

Serial con Optimización de Compilador y Memoria						
Rep.	N=8K	N=64K	N=500K	N=2M	N=5M	N=10M
1	0,387	2,754	22,467	91,198	228,620	455,020
2	0,360	2,747	22,433	90,059	226,004	454,266
3	0,359	2,760	22,596	89,730	228,227	454,143
4	0,457	2,748	22,437	90,052	226,365	448,395
5	0,366	2,757	22,618	89,859	225,567	455,128
6	0,365	2,753	22,570	90,094	228,372	452,826
7	0,365	2,761	22,563	89,807	225,392	453,468
8	0,360	2,754	22,512	89,940	228,635	455,560
9	0,362	2,755	22,469	89,870	228,844	454,098
10	0,359	2,750	22,485	90,126	225,122	448,253
Promedio	0,374	2,754	22,515	90,073	227,115	453,116
Desv. Est.	0,029	0,004	0,064	0,395	1,467	2,513
CV (%)	7,69	0,16	0,28	0,44	0,65	0,55

Cuadro 4: Tiempos de ejecución del algoritmo secuencial con optimización por compilador y memoria

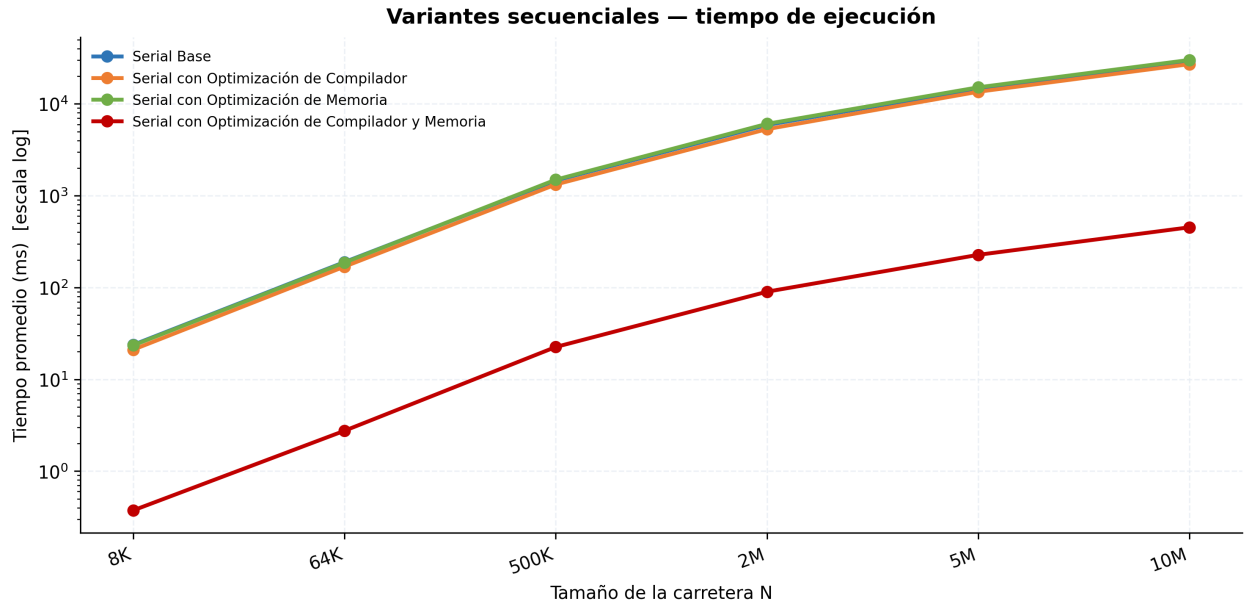


Figura 2: Gráfica de tiempos de los algoritmos secuenciales

Tabla 2 — Tiempo promedio y Speedup (referencia: Serial Base)

Implementación	N=8K Promedio (ms)	N=8K Speedup	N=64K Promedio (ms)	N=64K Speedup	N=500K Promedio (ms)	N=500K Speedup	N=2M Promedio (ms)	N=2M Speedup	N=5M Promedio (ms)	N=5M Speedup	N=10M Promedio (ms)	N=10M Speedup	Speedup Promedio
Serial Base	23,781	1,0000	189,390	1,0000	1.466,745	1,0000	5.899,681	1,0000	14.787,878	1,0000	29.608,895	1,0000	1,00
Serial con Optimización de Compilador	21,048	1,1298	168,952	1,1210	1.325,172	1,1068	5.311,727	1,1107	13.566,931	1,0900	27.118,113	1,0918	1,11
Serial con Optimización de Memoria	23,463	1,0136	186,912	1,0133	1.495,419	0,9808	6.069,491	0,9720	15.166,811	0,9750	30.145,522	0,9822	0,99
Serial con Optimización de Compilador y Memo	0,374	63,5858	2,754	68,7717	22,515	65,1452	90,073	65,4985	227,115	65,1119	453,116	65,3451	65,58

Cuadro 5: Speedup de las versiones secuenciales

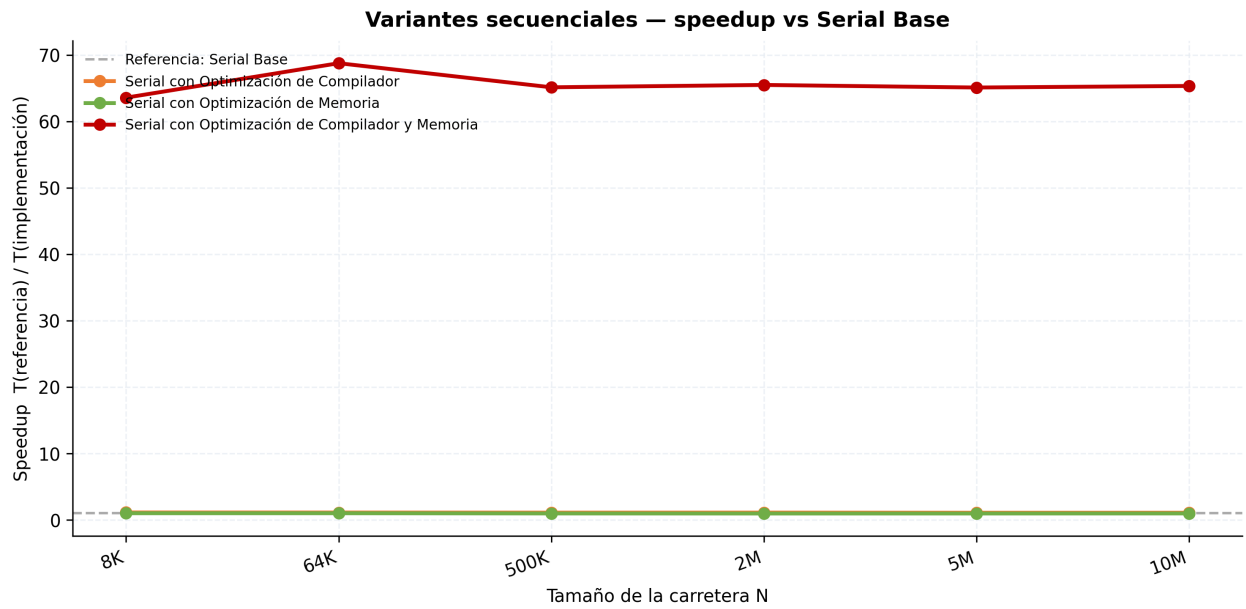


Figura 3: Gráfica de Speedup de los algoritmos secuenciales

4.2. Resultados de la implementación con MPI sin optimizaciones

Estos son los resultados obtenidos con la implementación con MPI sin optimizaciones ejecutado en las maquinas de AWS antes mencionadas.

MPI sin Optimización de Compilador (2 procesos)						
Rep.	N=8K	N=64K	N=500K	N=2M	N=5M	N=10M
1	29,285	223,465	1.721,056	6.924,939	17.585,710	22.504,684
2	28,174	225,430	1.728,623	7.151,027	17.544,976	35.355,844
3	29,098	223,345	1.756,268	7.153,642	17.414,119	34.726,192
4	27,474	220,935	1.733,215	6.884,214	17.250,233	34.853,386
5	27,617	223,354	1.742,095	6.935,596	17.545,018	34.577,984
6	27,673	221,328	1.752,484	6.990,071	17.654,970	35.034,279
7	29,445	221,411	1.739,595	6.955,120	17.732,362	34.967,484
8	27,603	227,036	1.774,644	7.082,769	17.459,315	34.770,893
9	30,243	220,453	1.758,250	6.950,895	17.556,084	23.428,245
10	28,953	225,621	1.759,119	6.975,124	17.351,426	35.174,976
Promedio	28,556	223,238	1.746,535	7.000,340	17.509,421	32.539,397
Desv. Est.	0,922	2,122	15,592	90,330	136,070	4.795,615
CV (%)	3,23	0,95	0,89	1,29	0,78	14,74

Cuadro 6: Tiempos de ejecución de la implementación con MPI con 2 procesos

MPI sin Optimización de Compilador (3 procesos)						
Rep.	N=8K	N=64K	N=500K	N=2M	N=5M	N=10M
1	927,749	929,480	1.354,633	3.378,870	7.646,036	23.428,281
2	926,666	933,274	1.374,200	4.711,170	11.812,673	23.373,971
3	931,387	927,710	1.365,419	4.772,398	11.577,951	23.179,913
4	931,781	926,408	1.349,532	4.614,399	11.678,195	23.220,531
5	935,355	926,551	1.371,628	4.715,723	11.671,024	23.450,951
6	933,379	926,417	1.369,860	4.697,243	11.638,161	23.531,145
7	929,209	928,525	1.362,946	4.684,446	11.908,931	18.042,722
8	950,702	930,041	1.362,203	4.720,450	11.696,220	23.516,591
9	925,728	931,354	1.362,224	4.685,055	11.745,621	23.324,765
10	928,253	926,411	1.374,748	4.678,030	11.635,268	23.648,859
Promedio	932,021	928,617	1.364,739	4.565,778	11.301,008	22.871,773
Desv. Est.	6,863	2,272	7,831	397,435	1.221,693	1.615,342
CV (%)	0,74	0,24	0,57	8,70	10,81	7,06

Cuadro 7: Tiempos de ejecución de la implementación con MPI con 3 procesos

MPI sin Optimización de Compilador (4 procesos)						
Rep.	N=8K	N=64K	N=500K	N=2M	N=5M	N=10M
1	965,716	955,169	1.376,268	4.067,451	9.405,046	18.440,195
2	949,008	936,245	1.380,077	4.068,101	9.438,297	18.428,357
3	939,999	937,907	1.367,850	4.044,287	9.414,034	18.349,237
4	943,884	936,515	1.373,236	4.066,293	9.476,973	18.495,412
5	936,011	949,070	1.378,537	4.097,066	9.278,592	18.514,459
6	950,364	954,560	1.376,765	4.047,650	9.424,746	18.358,367
7	942,058	941,247	1.366,212	4.036,765	9.365,170	18.351,311
8	949,677	942,120	1.377,424	4.085,317	9.368,275	18.436,927
9	954,042	949,591	1.364,948	4.072,904	9.430,858	18.440,610
10	937,529	951,222	1.368,652	4.032,815	9.450,650	18.361,054
Promedio	946,829	945,365	1.372,997	4.061,865	9.405,264	18.417,593
Desv. Est.	8,460	7,008	5,306	19,964	53,316	57,218
CV (%)	0,89	0,74	0,39	0,49	0,57	0,31

Cuadro 8: Tiempos de ejecución de la implementación con MPI con 4 procesos

MPI sin Optimización de Compilador (5 procesos)						
Rep.	N=8K	N=64K	N=500K	N=2M	N=5M	N=10M
1	1.001,230	984,832	1.326,147	3.467,126	7.806,318	15.052,915
2	1.019,898	991,821	1.330,003	3.497,276	7.818,868	15.082,447
3	990,272	978,367	1.325,810	3.484,604	7.772,072	15.067,056
4	1.005,551	973,076	1.334,114	3.501,916	7.807,891	15.022,134
5	992,092	967,665	1.321,644	3.484,752	7.812,884	15.058,633
6	990,780	978,663	1.323,192	3.467,310	7.826,350	15.078,831
7	997,287	985,306	1.325,515	3.475,704	7.793,138	15.053,943
8	1.004,773	975,485	1.328,308	3.505,296	7.833,286	15.307,010
9	987,510	981,908	1.328,594	3.482,041	7.802,521	14.695,971
10	999,010	997,918	1.332,766	3.486,815	7.827,624	15.092,883
Promedio	998,840	981,504	1.327,609	3.485,284	7.810,095	15.051,182
Desv. Est.	9,188	8,475	3,754	12,566	17,332	140,095
CV (%)	0,92	0,86	0,28	0,36	0,22	0,93

Cuadro 9: Tiempos de ejecución de la implementación con MPI con 5 procesos

MPI sin Optimización de Compilador (6 procesos)						
Rep.	N=8K	N=64K	N=500K	N=2M	N=5M	N=10M
1	975,578	982,213	1.204,218	2.974,891	6.713,125	12.519,101
2	972,347	955,226	1.248,860	2.959,185	6.388,646	12.446,097
3	975,875	974,643	1.188,096	2.894,442	6.416,601	12.422,600
4	974,189	971,482	1.199,331	2.918,229	6.453,190	12.395,798
5	979,543	954,866	1.193,831	2.900,181	6.447,137	11.961,140
6	974,403	956,315	1.193,256	2.944,109	6.470,590	11.984,105
7	956,523	954,569	1.190,379	2.914,039	6.456,343	12.427,590
8	979,118	966,421	1.192,750	2.923,561	6.433,425	12.405,905
9	981,599	968,193	1.171,521	2.905,773	6.452,940	12.418,045
10	975,768	968,390	1.189,037	2.919,829	6.455,042	12.414,652
Promedio	974,494	965,232	1.197,128	2.925,424	6.468,704	12.339,503
Desv. Est.	6,555	9,148	19,039	24,770	84,503	186,318
CV (%)	0,67	0,95	1,59	0,85	1,31	1,51

Cuadro 10: Tiempos de ejecución de la implementación con MPI con 6 procesos

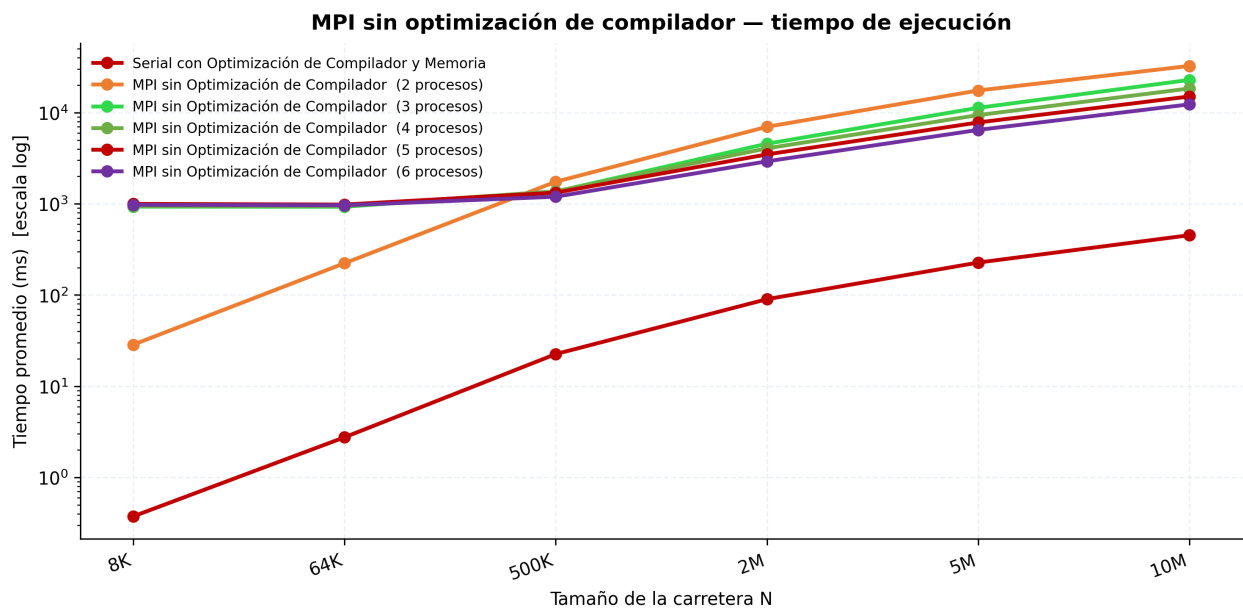


Figura 4: Gráfica de tiempo de la implementación con MPI sin optimizaciones

Tabla 2 — Tiempo promedio y Speedup (referencia: Serial con Optimización de Compilador y Memoria)												
Implementación	N=8K Promedio (ms)	N=8K Speedup	N=64K Promedio (ms)	N=64K Speedup	N=500K Promedio (ms)	N=500K Speedup	N=2M Promedio (ms)	N=2M Speedup	N=5M Promedio (ms)	N=5M Speedup	N=10M Promedio (ms)	N=10M Speedup
Serial con Optimización de Compilador y Memoria	0,374	1,0000	2,754	1,0000	22,515	1,0000	90,073	1,0000	227,115	1,0000	453,116	1,0000
MPI sin Optimización de Compilador (2 procesos)	28,556	0,0131	223,238	0,0123	1.746,535	0,0129	7.000,340	0,0129	17.509,421	0,0130	32.539,397	0,0139
MPI sin Optimización de Compilador (3 procesos)	932,021	0,0004	928,617	0,0030	1.364,739	0,0165	4.565,778	0,0197	11.301,008	0,0201	22.871,773	0,0198
MPI sin Optimización de Compilador (4 procesos)	946,829	0,0004	945,365	0,0029	1.372,997	0,0164	4.061,865	0,0222	9.405,264	0,0241	18.417,593	0,0246
MPI sin Optimización de Compilador (5 procesos)	998,840	0,0004	981,504	0,0028	1.327,609	0,0170	3.485,284	0,0258	7.810,095	0,0291	15.051,182	0,0301
MPI sin Optimización de Compilador (6 procesos)	974,494	0,0004	965,232	0,0029	1.197,128	0,0188	2.925,424	0,0308	6.468,704	0,0351	12.339,503	0,0367

Cuadro 11: Speedup de la implementación con MPI sin optimizaciones

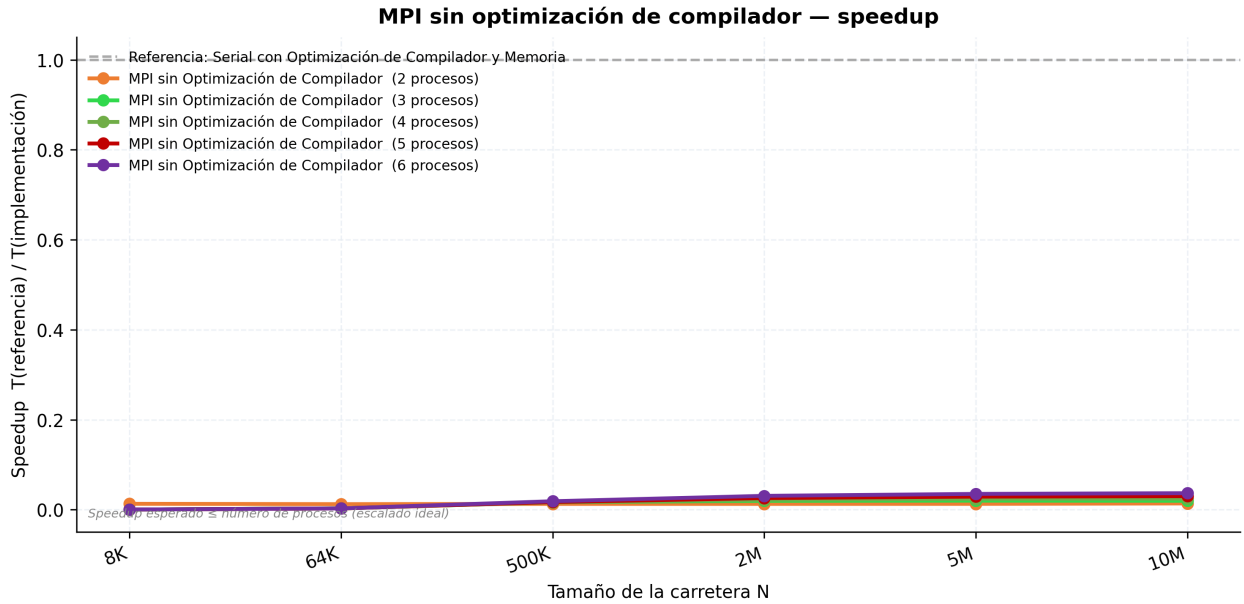


Figura 5: Gráfica de Speedup de la implementación con MPI sin optimizaciones

4.3. Resultados de la implementación con MPI con optimizaciones por compilador

Estos son los resultados obtenidos con la implementación con MPI con optimización por compilador usando las flags mencionadas en la sección 4 ejecutado en las maquinas de AWS antes mencionadas.

Serial con Optimización de Compilador y Memoria						
Rep.	N=8K	N=64K	N=500K	N=2M	N=5M	N=10M
1	0,387	2,754	22,467	91,198	228,620	455,020
2	0,360	2,747	22,433	90,059	226,004	454,266
3	0,359	2,760	22,596	89,730	228,227	454,143
4	0,457	2,748	22,437	90,052	226,365	448,395
5	0,366	2,757	22,618	89,859	225,567	455,128
6	0,365	2,753	22,570	90,094	228,372	452,826
7	0,365	2,761	22,563	89,807	225,392	453,468
8	0,360	2,754	22,512	89,940	228,635	455,560
9	0,362	2,755	22,469	89,870	228,844	454,098
10	0,359	2,750	22,485	90,126	225,122	448,253
Promedio	0,374	2,754	22,515	90,073	227,115	453,116
Desv. Est.	0,029	0,004	0,064	0,395	1,467	2,513
CV (%)	7,69	0,16	0,28	0,44	0,65	0,55

Cuadro 12: Tiempos de ejecución de la implementación con MPI con 2 procesos

MPI con Optimización de Compilador (2 procesos)						
Rep.	N=8K	N=64K	N=500K	N=2M	N=5M	N=10M
1	27,969	78,593	651,934	2.573,844	6.356,066	12.647,730
2	11,124	81,153	655,208	2.545,460	6.317,552	12.954,894
3	11,644	80,076	645,895	2.498,567	6.358,649	12.863,229
4	10,796	79,506	630,080	2.612,930	6.420,172	12.676,161
5	11,565	80,952	658,044	2.582,284	6.400,832	12.811,581
6	10,858	82,161	647,142	2.549,102	6.456,516	12.764,618
7	11,386	78,743	623,445	2.590,930	6.401,273	12.726,391
8	11,367	80,203	628,830	2.569,284	6.321,509	12.748,508
9	11,251	79,371	663,553	2.555,673	6.390,076	13.038,374
10	11,076	81,021	646,301	2.543,208	6.443,448	12.790,794
Promedio	12,904	80,178	645,043	2.562,128	6.386,609	12.802,228
Desv. Est.	5,029	1,090	12,737	29,820	45,161	115,250
CV (%)	38,97	1,36	1,97	1,16	0,71	0,90

Cuadro 13: Tiempos de ejecución de la implementación con MPI con 3 procesos

MPI con Optimización de Compilador (4 procesos)						
Rep.	N=8K	N=64K	N=500K	N=2M	N=5M	N=10M
1	949,275	947,658	947,218	1.735,055	3.517,600	6.720,790
2	959,139	942,012	963,724	1.728,439	3.572,181	6.868,866
3	942,594	956,074	932,004	1.732,161	3.557,735	6.626,775
4	951,531	949,934	946,034	1.743,324	3.610,715	6.906,881
5	938,713	935,784	954,882	1.747,604	3.596,878	6.628,112
6	962,211	939,330	950,173	1.717,007	3.558,788	6.680,524
7	941,340	944,878	948,067	1.757,581	3.525,885	6.650,403
8	957,655	951,240	940,817	1.751,412	3.567,506	6.652,729
9	952,451	938,776	941,761	1.722,495	3.625,649	6.636,681
10	952,833	959,365	947,175	1.699,099	3.536,015	6.732,910
Promedio	950,774	946,505	947,185	1.733,418	3.566,895	6.710,467
Desv. Est.	7,479	7,358	8,027	16,741	33,933	95,576
CV (%)	0,79	0,78	0,85	0,97	0,95	1,42

Cuadro 14: Tiempos de ejecución de la implementación con MPI con 4 procesos

MPI con Optimización de Compilador (5 procesos)						
Rep.	N=8K	N=64K	N=500K	N=2M	N=5M	N=10M
1	994,357	983,554	1.042,671	1.609,444	3.060,045	5.478,189
2	1.016,487	1.013,978	1.052,875	1.596,705	3.107,688	5.539,258
3	978,824	1.012,784	1.045,237	1.594,903	3.074,568	5.546,669
4	1.002,788	980,366	1.045,298	1.607,956	3.061,403	5.469,019
5	997,976	972,589	1.056,759	1.604,131	3.069,162	5.481,905
6	998,244	1.017,709	1.053,707	1.611,245	3.091,880	5.539,605
7	989,114	1.005,457	1.047,474	1.622,669	3.064,493	5.546,217
8	996,240	1.014,481	1.056,644	1.623,251	3.064,020	5.485,443
9	988,144	986,181	1.054,743	1.599,625	3.111,517	5.925,647
10	1.010,105	991,337	1.052,988	1.615,798	3.119,287	5.494,849
Promedio	997,228	997,844	1.050,840	1.608,573	3.082,406	5.550,680
Desv. Est.	10,322	15,933	4,910	9,498	21,848	128,420
CV (%)	1,04	1,60	0,47	0,59	0,71	2,31

Cuadro 15: Tiempos de ejecución de la implementación con MPI con 5 procesos

MPI con Optimización de Compilador (6 procesos)						
Rep.	N=8K	N=64K	N=500K	N=2M	N=5M	N=10M
1	982,449	975,698	1.020,444	1.426,122	2.657,721	4.700,174
2	971,049	973,255	1.016,061	1.413,191	2.600,532	4.585,937
3	975,960	982,505	1.018,308	1.411,383	2.572,167	4.620,583
4	985,923	986,656	1.017,797	1.417,557	2.592,506	4.588,141
5	981,646	970,049	1.019,631	1.432,454	2.615,946	4.679,423
6	954,840	957,949	1.016,235	1.424,390	2.691,716	4.661,904
7	969,303	987,772	1.018,058	1.418,837	2.616,316	4.570,302
8	979,094	980,276	1.022,517	1.408,997	2.581,729	4.643,011
9	974,409	983,532	1.011,940	1.402,661	2.601,243	4.618,695
10	978,427	950,597	1.012,996	1.402,313	2.615,429	4.531,214
Promedio	975,310	974,829	1.017,399	1.415,791	2.614,530	4.619,938
Desv. Est.	8,380	11,704	3,069	9,472	33,996	49,774
CV (%)	0,86	1,20	0,30	0,67	1,30	1,08

Cuadro 16: Tiempos de ejecución de la implementación con MPI con 6 procesos

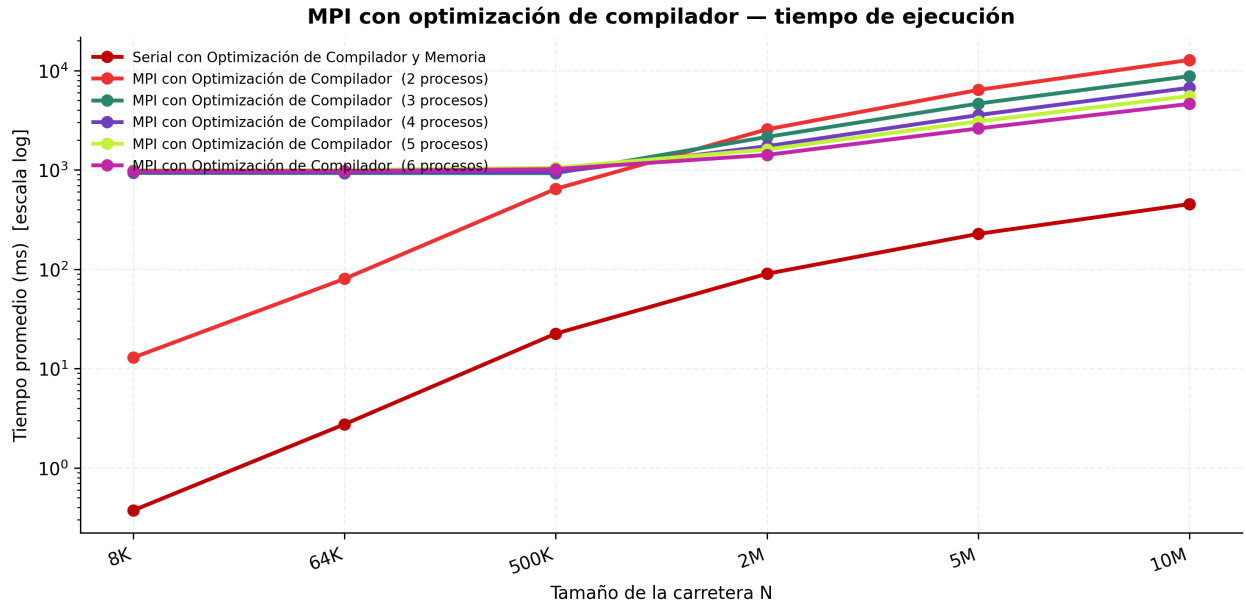


Figura 6: Gráfica de tiempos de la implementación con MPI con optimización

Tabla 2 — Tiempo promedio y Speedup (referencia: Serial con Optimización de Compilador y Memoria)

Implementación	N=8K Promedio (ms)	N=8K Speedup	N=64K Promedio (ms)	N=64K Speedup	N=500K Promedio (ms)	N=500K Speedup	N=2M Promedio (ms)	N=2M Speedup	N=5M Promedio (ms)	N=5M Speedup	N=10M Promedio (ms)	N=10M Speedup	Speedup Promedio
Serial con Optimización de Compilador y Memoria	0,374	1,0000	2,754	1,0000	22,515	1,0000	90,073	1,0000	227,115	1,0000	453,116	1,0000	1,00
MPI con Optimización de Compilador (2 procesos)	12,904	0,0290	80,178	0,0343	645,043	0,0349	2.562,128	0,0352	6.386,609	0,0356	12.802,228	0,0354	0,03
MPI con Optimización de Compilador (3 procesos)	931,751	0,0004	929,500	0,0030	927,838	0,0243	2.148,997	0,0419	4.657,298	0,0488	8.794,823	0,0515	0,03
MPI con Optimización de Compilador (4 procesos)	950,774	0,0004	946,505	0,0029	947,185	0,0238	1.733,418	0,0520	3.566,895	0,0637	6.710,467	0,0675	0,04
MPI con Optimización de Compilador (5 procesos)	997,228	0,0004	997,844	0,0028	1.050,840	0,0214	1.608,573	0,0560	3.082,406	0,0737	5.550,680	0,0816	0,04
MPI con Optimización de Compilador (6 procesos)	975,310	0,0004	974,829	0,0028	1.017,399	0,0221	1.415,791	0,0636	2.614,530	0,0869	4.619,938	0,0981	0,05

Cuadro 17: Speedup de la implementación con MPI con optimización

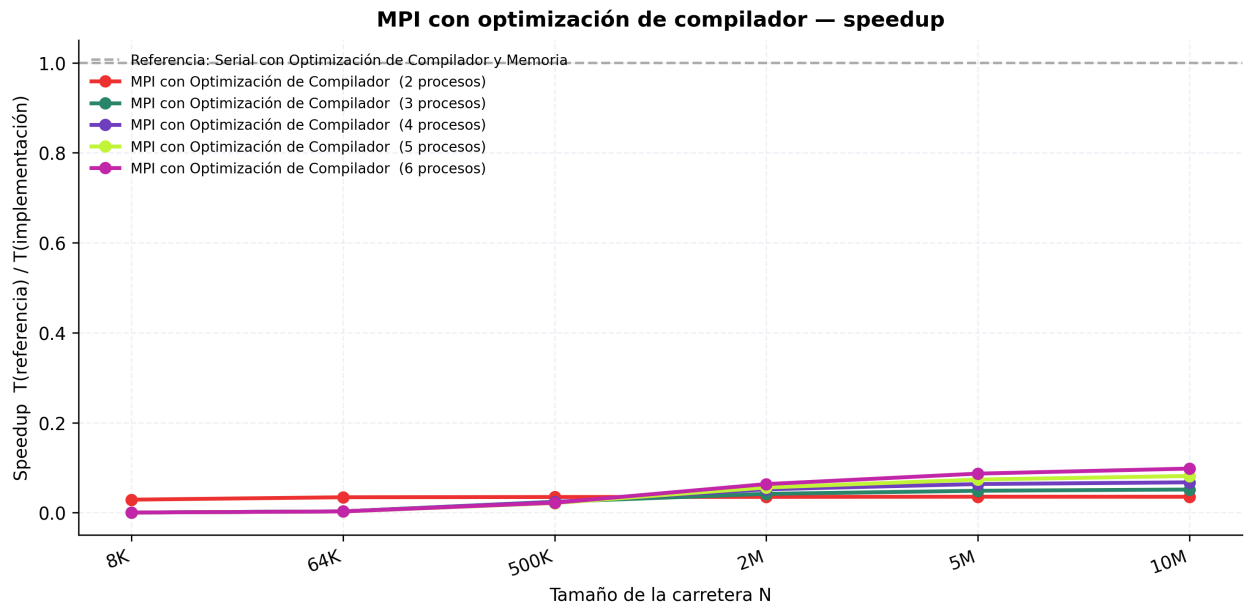


Figura 7: Gráfica de Speedup de la implementación con MPI con optimización

4.4. Comparación

Se muestran las gráficas de comparación de la mejor variante de cada implementación.

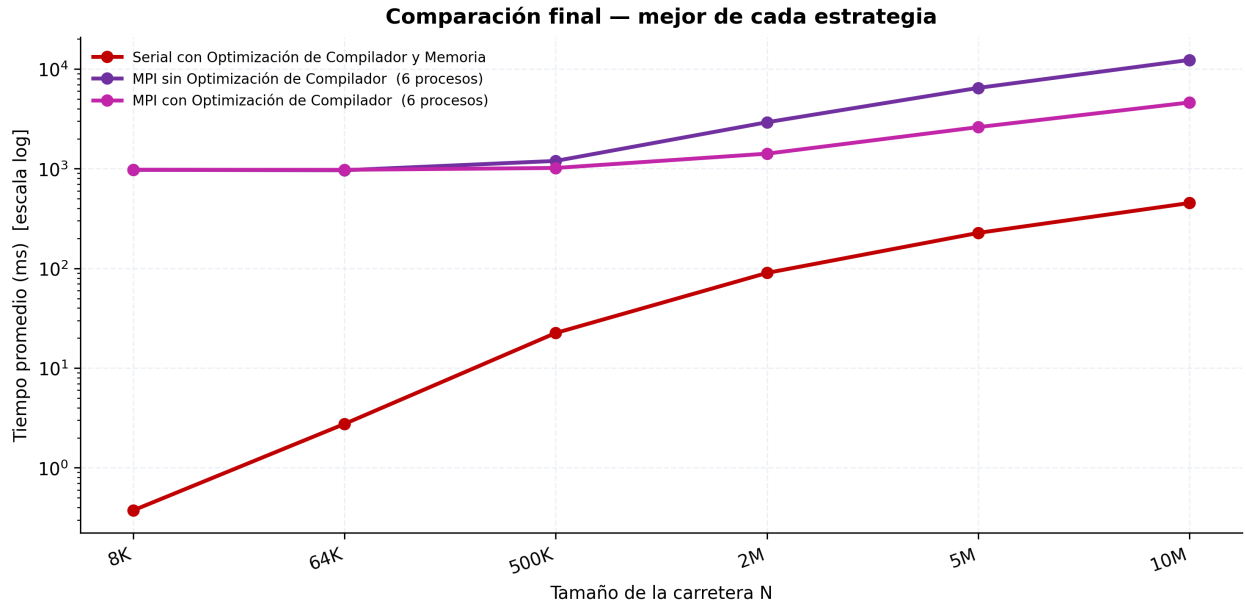


Figura 8: Comparación del tiempo de las mejores variantes

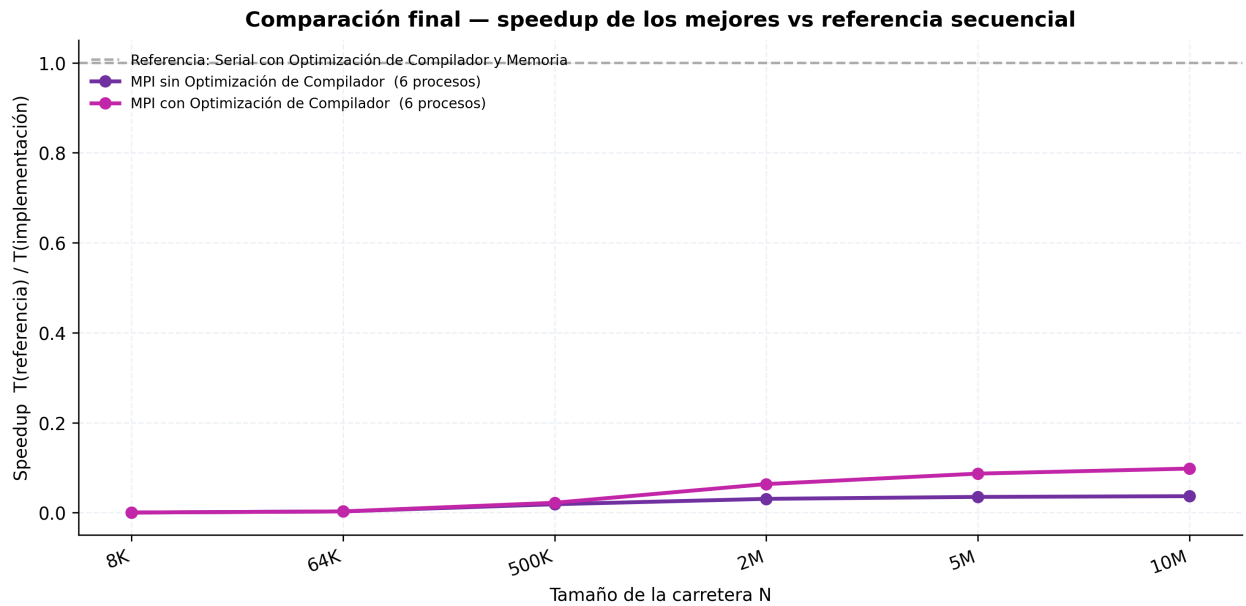


Figura 9: Comparación del Speedup de las mejores variantes

5. Conclusiones

1. La primera observación relevante surge al comparar las cuatro variantes secuenciales evaluadas. Los resultados obtenidos fueron cercanos a lo que se vio en el Reto 2, mientras que la optimización de compilador por sí sola produce una mejora modesta, cercana al 10 %, y la optimización de memoria aislada incluso introduce una ligera regresión de rendimiento, la combinación de ambas genera una aceleración extraordinaria. La versión que incorpora simultáneamente optimización de compilador y alineación de memoria alcanza speedups promedio cercanos a $65\times$ respecto a la implementación base.

El análisis del porqué esto ocurre se encuentra en nuestro trabajo anterior.

2. Al comparar la mejor versión secuencial optimizada con las implementaciones MPI compiladas sin optimizaciones, se observa que el paralelismo distribuido no logra compensar los costos adicionales introducidos por la comunicación entre procesos. Para tamaños pequeños, como $N = 8\,000$, la versión MPI con dos procesos resulta aproximadamente 76 veces más lenta que la referencia secuencial optimizada.

La causa principal es que el tiempo invertido en inicialización, distribución de datos, intercambio de halos y operaciones colectivas domina completamente el tiempo de cómputo local. Dado que el trabajo realizado por cada proceso es muy reducido, el costo de comunicación se convierte en el factor predominante. A medida que aumenta el tamaño del problema, especialmente para $N \geq 2$ millones de celdas, comienza a apreciarse una reducción progresiva de los tiempos al incrementar el número de procesos, lo que indica que el paralelismo empieza a amortizar parcialmente el overhead de MPI.

3. La incorporación de las mismas optimizaciones utilizadas en la versión secuencial mejora significativamente el desempeño de las implementaciones MPI. El caso más evidente es el de dos procesos, cuyo tiempo para $N = 8\,000$ disminuye de aproximadamente 28.6 ms a 12.9 ms.

Sin embargo, aun con estas mejoras, la versión paralela continúa siendo considerablemente más lenta que la secuencial optimizada. Para cargas pequeñas, la diferencia sigue siendo superior a un orden de magnitud. Además, se observa una alta variabilidad en las mediciones para configuraciones ligeras, alcanzando coeficientes de variación cercanos al 39 %. Este comportamiento es consistente con la naturaleza compartida de la infraestructura en la nube y con la sensibilidad de las aplicaciones de baja carga a fluctuaciones de latencia en la red.

En tamaños grandes la situación mejora progresivamente. Por ejemplo, con $N = 10$ millones de celdas y seis procesos, el tiempo se reduce hasta aproximadamente 4.6

segundos. Aunque esta cifra sigue siendo superior a la obtenida por la versión secuencial optimizada, la tendencia evidencia que el paralelismo comienza a resultar beneficioso cuando el volumen de trabajo crece suficientemente.

4. La comparación final muestra que el rendimiento no depende únicamente del número de procesos, sino de la combinación entre paralelismo y optimización de compilación. Un resultado particularmente interesante es que una configuración MPI sin optimización pero con seis procesos alcanza tiempos similares a los obtenidos por una configuración MPI optimizada con únicamente dos procesos.

Esto demuestra que aumentar el número de procesos puede compensar parcialmente la ausencia de optimizaciones de compilación. No obstante, la mejor estrategia sigue siendo combinar ambas técnicas, ya que la optimización del código reduce significativamente el trabajo realizado por cada proceso y permite aprovechar mejor los recursos disponibles.

5. De manera general, los resultados muestran que MPI sin optimización de compilación no es una alternativa eficiente para este problema en las instancias evaluadas. El overhead de comunicación supera sistemáticamente el beneficio del paralelismo cuando el tamaño del problema es pequeño o moderado.

Aun así, sí se observa un punto de mejora conforme crece el tamaño de la carretera. Esto sugiere que el paralelismo comienza a volverse competitivo solo cuando el trabajo por proceso es suficientemente grande como para amortizar el costo de comunicación. En otras palabras, el cruce entre la versión secuencial optimizada y la versión MPI depende fuertemente del tamaño del problema y todavía no se alcanza en los casos más pequeños.

6. El principal cuello de botella de la implementación distribuida no es el volumen de datos intercambiados, sino la latencia asociada a las operaciones de comunicación y sincronización. Los halos transmitidos son mínimos, de modo que el costo dominante proviene del tiempo que toma coordinar los procesos y esperar a que todos avancen al mismo ritmo.

Por eso, aunque la estrategia de solapamiento mediante `MPI_Irecv` y `MPI_Isend` es correcta desde el punto de vista algorítmico, su beneficio resulta limitado para cargas pequeñas y medianas. El cómputo interior del bloque local no siempre alcanza para esconder de forma efectiva la latencia de red, y la mejora real solo se vuelve visible cuando el tamaño del problema es lo bastante grande.

7. Al analizar la escalabilidad de la implementación MPI, es necesario considerar primero las características del clúster utilizado. Las pruebas se ejecutaron sobre tres instancias

EC2 tipo `m7i-flex.large`: un nodo principal y dos nodos de trabajo. Cada instancia dispone de 2 vCPUs, equivalentes a un núcleo físico con dos hilos mediante Hyper-Threading. Debido a esta configuración, el comportamiento del algoritmo no depende únicamente del número de procesos MPI, sino también de cómo estos se distribuyen entre los nodos físicos. Cuando la ejecución requiere comunicación entre nodos diferentes, aparecen costos asociados a la red que no están presentes cuando los procesos comparten memoria dentro de una misma máquina.

Los resultados muestran que el cambio más significativo ocurre al pasar de configuraciones que pueden ejecutarse localmente dentro de un nodo a configuraciones que requieren comunicación distribuida. Para tamaños pequeños, como $N = 8\,000$, el tiempo pasa de aproximadamente 28.6 ms con dos procesos a cerca de 932 ms con tres procesos. Este incremento no corresponde a una degradación progresiva del algoritmo, sino a la aparición de un nuevo costo dominante: la sincronización entre nodos a través de la red.

En este problema cada iteración requiere intercambios de halos y operaciones colectivas como `MPI_Allreduce`. Cuando el trabajo computacional por proceso es muy pequeño, el tiempo dedicado a estas comunicaciones supera ampliamente el tiempo invertido en actualizar las celdas del autómata. Como consecuencia, la latencia de la red termina dominando el tiempo total de ejecución.

8. Una vez que la ejecución ya involucra múltiples nodos, aumentar el número de procesos sí produce mejoras de rendimiento, especialmente para los tamaños de carretera más grandes. En el caso de $N = 10$ millones de celdas con optimización de compilación, el tiempo disminuye progresivamente al pasar de tres a seis procesos, alcanzando una aceleración cercana a $2,8\times$ respecto a la configuración de dos procesos.

Aunque esta mejora demuestra que el trabajo se distribuye correctamente entre los procesos disponibles, el speedup obtenido permanece por debajo del ideal teórico.

Referencias

- Ben-Ari, M. (2006). *Principles of Concurrent and Distributed Programming* (2nd). Addison-Wesley.
- Drepper, U. (2007). *What Every Programmer Should Know About Memory* (Technical Report). Red Hat, Inc. <https://people.freebsd.org/~lstewart/articles/cpumemory.pdf>
- Gropp, W., Lusk, E., & Skjellum, A. (2014). *Using MPI: Portable Parallel Programming with the Message-Passing Interface* (3.^a ed.). MIT Press.
- Gustafson, J. L. (1988). Reevaluating Amdahl's Law. *Communications of the ACM*, 31(5), 532-533. <https://doi.org/10.1145/42411.42415>
- Hager, G., & Wellein, G. (2010). *Introduction to High Performance Computing for Scientists and Engineers*. CRC Press.
- MPI Forum. (2021). MPI: A Message-Passing Interface Standard, Version 4.0 [Accessed 2026-05-26].
- Nagel, K., & Schreckenberg, M. (1992). A cellular automaton model for freeway traffic. *Journal de Physique I*, 2(12), 2221-2229. <https://doi.org/10.1051/jp1:1992277>
- Pacheco, P. S. (1997). *Parallel Programming with MPI*. Morgan Kaufmann.
- Patterson, D. A., & Hennessy, J. L. (2017). *Computer Architecture: A Quantitative Approach* (6th). Morgan Kaufmann.
- Sandberg, R., Goldberg, D., Kleiman, S., Walsh, D., & Lyon, B. (1985). Design and Implementation of the Sun Network Filesystem. *USENIX Conference Proceedings*.
- Silberschatz, A., Galvin, P. B., & Gagne, G. (2018). *Operating System Concepts* (10th). Wiley.
- Snir, M., Otto, S., Huss-Lederman, S., Walker, D., & Dongarra, J. (1998). *MPI: The Complete Reference, Volume 1: The MPI Core* (2.^a ed.). The MIT Press.
- Stone, H., & Hennessy, J. L. (1995). Performance of Network File Systems. *IEEE Transactions on Computers*.
- Stoner, J. C., Terry, M., et al. (1995). The Design and Implementation of a Network File System. *ACM Operating Systems Review*.
- Storm, R. L., et al. (1988). NFS and the Distributed File System Paradigm. *IEEE Computer*.
- Tanenbaum, A. S., & Steen, M. V. (2015). *Distributed Systems: Principles and Paradigms* (2.^a ed.). Pearson.
- Terry, D. B. (1995). Session Guarantees for Weakly Consistent Replicated Data. *ACM SIGOPS Operating Systems Review*.

- Williams, S., Waterman, A., & Patterson, D. A. (2009). Roofline: an insightful visual performance model for multicore architectures. *Communications of the ACM*, 52(4), 65-76. <https://doi.org/10.1145/1498765.1498785>
- Wolfram, S. (1984). Universality and complexity in cellular automata. *Physica D: Nonlinear Phenomena*, 10(1-2), 1-35. [https://doi.org/10.1016/0167-2789\(84\)90245-8](https://doi.org/10.1016/0167-2789(84)90245-8)
- Yi, L., Li, C., & Guo, J. (2020). CPI for Runtime Performance Measurement: The Good, the Bad, and the Ugly. *IEEE International Symposium on Workload Characterization (IISWC 2020)*, 15-25. <https://doi.org/10.1109/IISWC50251.2020.00011>